GenSpark

OOP principles

So far you have organized programs with variables, functions, and files. That works well until a program has many pieces of data that belong together and many functions that depend on them. The Number Guessing Game already hinted at this: it tracked a secret number, a range, an attempt limit, and an attempt count, and every part of the game needed some of them. Object-oriented programming, or OOP, is a way of organizing code around **things** instead of around steps. This article explains the idea and its four main principles in plain terms. The next two articles show how to write classes in Python, then how to build families of related classes.

The Problem OOP Solves

Look at the data in your Week 1 guessing game. Six values describe one round: the low number, the high number, the secret number, the attempt limit, the attempts used, and whether the player has won. With only functions, you must pass those values in and send the changed ones back out:

python

```
def take_turn(guess, secret, attempts_used, max_attempts):
    attempts_used += 1
    result = judge_guess(guess, secret)
    remaining = max_attempts - attempts_used
    return result, attempts_used, remaining

result, attempts_used, remaining = take_turn(10, 42, 0, 8)
```

This has three weaknesses:

- **Long parameter lists.** Every function needs a handful of related values, and every call has to pass them all.
- **Scattered state.** The values live in separate variables. Nothing says they belong together.

- **No protection.** Any line of code can reach in and break a value. Nothing stops `attempts_used = -100`.

OOP solves this by putting the data and the functions that use it into one unit, called an **object**.

Objects: Data Plus Behavior

An **object** is a bundle of two things:

- **State**: the data it remembers, called *attributes*.
- **Behavior**: the actions it can perform, called *methods*. A method is just a function that belongs to an object.

Real-world things work this way. A thermostat has state (the current temperature, the target temperature) and behavior (turn the heat on, turn it off, change the target). You do not reach inside a thermostat and move its wires. You use its controls.

Here is the guessing game round as an object. **Do not worry about every detail of the syntax.** The next article explains it line by line. Focus on the idea: the data and the functions that use it live together.

python

```
import random


class Game:
    def __init__(self, low, high, max_attempts, secret=None):
        self.low = low
        self.high = high
        self.max_attempts = max_attempts
        if secret is None:
            secret = random.randint(low, high)
        self.secret = secret
        self.attempts_used = 0

    def guess(self, number):
        self.attempts_used += 1
        if number < self.secret:
            return "too low"
```

```
        if number > self.secret:
            return "too high"
        return "correct"

    def attempts_remaining(self):
        return self.max_attempts - self.attempts_used

    def is_out_of_attempts(self):
        return self.attempts_used >= self.max_attempts
```

Using it looks like this:

python

```
game = Game(1, 100, 8, secret=42)

print(game.guess(10))              # too low
print(game.attempts_remaining())   # 7
print(game.guess(90))              # too high
print(game.guess(42))              # correct
print(game.attempts_remaining())   # 5
```

Compare this with `take_turn`. The caller no longer tracks the attempt count or passes the secret around. The `game` object remembers everything about its own round. You ask it to do things, and it keeps its own books.

Classes and Objects

The code above has a `class Game` and a `game` object. These are different:

Term	Meaning	Example
Class	A blueprint that describes what a kind of thing looks like and can do	`Game`
Object (or **instance**)	One actual thing built from the blueprint	`game = Game(1, 100, 8)`

One class can make many objects, and each keeps its own state:

python

```
easy = Game(1, 50, 10)
hard = Game(1, 500, 10)
```

These are two separate games with separate secrets and separate attempt counts. Changing one has no effect on the other. A blueprint for a house is not a house. You build as many houses as you like from it.

The Four Principles

OOP is usually described by four principles. Different books word them slightly differently, but these four are the ones to know.

1. Encapsulation

Encapsulation means bundling data with the code that works on it, and controlling how the outside world touches that data.

The `Game` object keeps `attempts_used` inside. The way to change it is to call `guess()`, which updates the count correctly every time. The caller does not have to remember to add one.

Encapsulation protects your data from accidental misuse. If only the object's own methods change `attempts_used`, then a bug that corrupts it can only be inside that one class. This also makes code easier to change later. If you decide to track attempts differently, you change the class, and nothing else in the program needs to know.

A note about Python: it does not strictly enforce privacy. You *can* still write `game.attempts_used = -5`, and Python lets you. Python relies on conventions, such as starting a name with an underscore to say "this is internal, leave it alone." The next article covers those conventions. Treat the protection as an agreement among programmers, not a lock.

2. Abstraction

Abstraction means showing only what is needed and hiding the rest. You use a car by pressing the pedals and turning the wheel. You do not need to know how fuel injection works.

When you call `game.guess(10)`, you do not need to know how the comparison is done or how the counter changes. You only need to know what `guess` accepts and what it returns. That set of

things you are allowed to use is called the object's **interface**.

Good abstraction lets you think at the right level. The code that runs the menus does not care how a win is detected. It only asks the game, "are we out of attempts?" You already did this with functions in the last article, where a function name stood in for a block of code. A class applies the same idea to a whole thing.

3. Inheritance

Inheritance lets one class be built on top of another, so the new class gets everything the old one has and adds or changes only what is different.

A timed version of the guessing game is still a guessing game. It just also tracks a clock. Instead of copying all the code, you say "a `TimedGame` is a kind of `Game` " and write only the new parts:

python

```
class TimedGame(Game):
    ...      # gets every attribute and method of Game, then adds a timer
```

Inheritance describes an **"is-a"** relationship. A `TimedGame` *is a* `Game` . An easy AI *is an* AI player. The Tic-Tac-Toe project uses it for the three AI levels. The article after next covers how to write it.

4. Polymorphism

Polymorphism means "many forms." Different kinds of objects can respond to the same request in their own way, so the code asking does not need to know which kind it has.

Imagine a Tic-Tac-Toe AI that comes in levels. Every level can be asked the same question, "choose a move", but each answers differently:

python

```
class EasyAI:
    def choose_move(self):
        return "picks a random cell"


class HardAI:
```

```
    def choose_move(self):
        return "searches every possible game"


for ai in [EasyAI(), HardAI()]:
    print(ai.choose_move())
```

Output:

```
picks a random cell
searches every possible game
```

The loop calls `choose_move()` and does not care which AI it has. That means the game code never needs `if level == "easy": ... elif level == "hard": ...`. You can add a fourth level later without changing the game at all. Polymorphism is what keeps long chains of `if` and `elif` out of your program.

The four together

Principle	One sentence	In Tic-Tac-Toe
Encapsulation	Keep data and its methods together, and guard the data	A `Board` controls how marks are placed
Abstraction	Expose a simple interface, hide the details	The game asks `board.has_won("X")` and does not care how
Inheritance	A new class builds on an existing one	`EasyAI`, `MediumAI`, and `HardAI` are kinds of AI player
Polymorphism	The same request works across different kinds	Every AI answers `choose_move()` its own way

"Is-a" and "Has-a"

When you design with classes, you decide how they relate. There are two main relationships:

Relationship	Question to ask	Example
Is-a (inheritance)	Is X a kind of Y?	A `HardAI` is an `AIPlayer`
Has-a (composition)	Does X contain a Y?	A `Game` has a `Board`

Composition means one object holds another as part of its state:

python

```
class Board:
    def __init__(self, size):
        self.size = size


class Match:
    def __init__(self, size):
        self.board = Board(size)        # a Match HAS a Board

match = Match(3)
print(match.board.size)                 # 3
```

A common beginner mistake is using inheritance whenever two classes are related in any way. A `Game` is not a kind of `Board`, so `Game(Board)` would be wrong. It merely uses one. A useful rule: **prefer composition, and use inheritance only when the "is-a" sentence is truly accurate.** Composition is more flexible, because the parts can be swapped without rewriting the whole.

Finding the Classes in a Problem

You do not have to guess what classes a program needs. Read the requirements and underline the **nouns** and the **verbs**:

- Nouns are candidates for classes, or for attributes of a class.
- Verbs are candidates for methods.

Take this requirement from the Tic-Tac-Toe lab: "Players take turns placing a mark on the board. The game detects a win or a draw. The computer opponent chooses its move based on its difficulty level."

Noun	Likely a...
board	Class
mark	A value stored on the board (`"X"` or `"O"`)
player	Possibly a class, possibly just a mark
game	Class
computer opponent	Class (with variants)
difficulty level	The differences between the AI variants
Verb	**Likely a method of...**
place a mark	Board
detect a win, detect a draw	Board
take turns	Game
choose a move	AI player

From that, a first design looks like this:

Class	Holds (data)	Does (methods)
`Board`	The grid, the size	Place a mark, list empty cells, detect a win, detect a draw
`AIPlayer`	Its mark	Choose a move
`EasyAI` , `MediumAI` , `HardAI`	Their mark	Each chooses a move its own way
`Game`	A board, an AI, whose turn it is, the time limit	Take a turn, handle a forfeit, report when the game is over

And the relationships, as a simple outline:

```
Game
  has a Board
  has an AIPlayer (which may be an EasyAI, MediumAI, or HardAI)

AIPlayer
  is the base of EasyAI, MediumAI, and HardAI
```

This is the same layout you saw in the functions article: `board.py`, `ai.py`, `game.py`, and `main.py`. Menus and printing stay in `main.py`, outside the classes, so the game logic can be tested without a person typing. A rough design like this one takes ten minutes and saves hours.

Principles for Designing Classes

One responsibility per class. A class should have one reason to exist. A `Board` that also prints menus, asks for input, and runs the AI has too many jobs. If you cannot describe a class in one sentence without "and", split it.

Name classes with nouns and methods with verbs. Class names use `CamelCase`: `Board`, `TimedGame`. Methods use `snake_case`, like variables: `place_mark`, `choose_move`.

Keep the interface small. Expose only what other code needs. A small interface is easier to learn, test, and change.

Keep logic apart from input and output. The same rule as in the functions article. Classes decide and remember. A thin outer layer prints and reads. This is what lets you test a class with pytest.

Don't repeat yourself. If two classes share the same code, move it into a shared base class or a helper function.

Keep it simple. The KISS principle from the coding best practices article applies. A design with five small, clear classes beats one with fifteen clever ones.

When to Use OOP, and When Not To

OOP is a tool, not a rule. Python lets you mix styles freely.

Use classes when:

- You have **things with both state and behavior**, like a board, a game, an account, or a

connection.

- You have **several variations of one idea**, like AI levels or payment types, where polymorphism helps.
- You want to **test parts separately** and swap them out.
- The program is **large enough** that organization matters.

Plain functions are fine when:

- The task is a **short script**, like most of Week 1.
- You are **transforming data** from input to output with no state to remember.
- A class would only be a container for one function.

A frequent beginner error is turning everything into a class. If a class has only an `__init__` and a single method, it is probably a function in disguise.

Common Mistakes

Mistake	What happens	Fix
Confusing a class with an object	You try to use the blueprint like a finished thing	A class is the pattern. Create an object from it.
Making everything a class	Extra code with no benefit	Use functions for simple tasks
One giant class that does everything	Hard to read, test, or change	Split by responsibility
Using inheritance for "uses" relationships	A `Game` pretending to be a `Board`	Use composition (has-a)
Reaching into an object to change its data directly	Bugs that are hard to trace	Change state through methods
Printing and asking for input inside logic classes	Cannot be tested without typing	Keep I/O in `main.py`
Giving every class public,		

changeable details	No protection	Expose a small interface
Designing the whole system before writing anything	Over-planning	Sketch the nouns and verbs, then build one class at a time
Naming classes like functions	`place_board` is confusing	Classes are nouns: `Board`. Methods are verbs: `place_mark`.

Try It Yourself

Most of these exercises are about design, not code. Write the answers in a text file or on paper.

1. For each item, say whether it is better modeled as a function or a class, and why: converting Celsius to Fahrenheit, a vending machine, a bank account, adding two numbers, a playing card deck.
2. Read this requirement and underline the nouns and verbs: "A library lets members borrow books. Each book has a title and an author. Members can borrow up to three books at a time. Overdue books are fined."
3. From exercise 2, build a table of classes with what each one holds and what each one does.
4. For these pairs, say whether the relationship is "is-a" or "has-a": a car and an engine, a dog and an animal, a library and a book, a square and a shape, a game and a timer.
5. Describe the thermostat from this article as a class: list its attributes and its methods.
6. Take the six values in the guessing game round (low, high, secret, max attempts, attempts used, won). Write a one-sentence reason why they belong in one object instead of six separate variables.
7. Type the `Game` class from this article into a file and run the usage example. Create a second game with a different range, and show that the two keep separate attempt counts.
8. Add a method to `Game` called `is_won` that returns `True` if the last guess was correct. (You will need to remember the result of the last guess in an attribute.)
9. In your `Game` object, set `game.attempts_used = -5` and call

`attempts_remaining()`. What does Python do? What does this tell you about encapsulation in Python?

10. Type in the `EasyAI` and `HardAI` classes and the loop. Add a `MediumAI` class with its own `choose_move`. Does the loop need to change?

11. Sketch a design for a text adventure game on paper: a player moves between rooms, picks up items, and tries to escape. List the classes, what each holds, and what each does.

12. Pick a Week 1 program of yours (the Number Guessing Game works well). Write the list of classes you would use if you rebuilt it with OOP, with one sentence each.

Why This Matters for an FDE

Most client codebases you will join are object-oriented, whether they are in Python, Java, or C#. Reading them means understanding how responsibilities were split into classes and how those classes fit together. When you build software for a client, the structure you choose decides whether it is easy to extend six months later. OOP is also the vocabulary your team uses: when someone says "that should be its own class" or "use composition here", they are applying the ideas in this article. The AI agent frameworks you will use later in the program are built from classes as well.

Comprehension Check

1. What are the two things an object bundles together? Which Python words describe each one?

2. What is the difference between a class and an object? Give an example from the guessing game.

3. In your own words, explain encapsulation. Why does Python's lack of strict privacy mean we rely on agreement among programmers?

4. What is an interface, and how does abstraction relate to it?

5. A `TimedGame` is built on a `Game`. Which OOP principle is this, and what is the relationship called?

6. What does polymorphism let the Tic-Tac-Toe game avoid writing? Give the specific code it spares you.

7. For each pair, say "is-a" or "has-a": `HardAI` and `AIPlayer`, `Game` and `Board`.
8. How do you find candidate classes and methods in a set of requirements?
9. Why is it a design mistake for a `Board` class to ask the player for input?
10. Name one situation where plain functions are a better choice than classes.

← Previous: Lists and Nested Lists (needed for the Tic-Tac-Toe board)

Next: OOP: Classes, Objects & Encapsulation →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!